

1. Factorial of given num and sum of fact series.

SERVER CODE:

```
import socket

import math

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

host = "localhost"

port = 12345

server.bind((host, port))

server.listen(1)

print("Server waiting for connection...")

conn, addr = server.accept()

print("Connected by:", addr)

number = int(conn.recv(1024).decode())

print("Received number:", number)

# Factorial of given number

factorial = math.factorial(number)

# Sum of factorial series

sum_fact = 0

series = ""

for i in range(1, number + 1):

    fact = math.factorial(i)

    sum_fact += fact

    series += str(fact) + " "

# Send both results

result = "Factorial Series = " + series + "\nFactorial = " + str(factorial) + "\nSum of Factorial Series = " + str(sum_fact)

# result = "Factorial = " + str(factorial) + "\nSum of Factorial Series = " + str(series) + str(sum_fact)

conn.send(result.encode())

conn.close()

server.close()
```

CLIENT CODE:

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

host = "localhost"

port = 12345

client.connect((host, port))

num = input("Enter a number: ")
```

```
client.send(num.encode())

result = client.recv(1024).decode()

print(result)

client.close()
```

2. Given number is Palindrome or not. Check whether the palindrome number is even or odd

SERVER CODE:

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server_socket.bind(("localhost", 18000))
server_socket.listen(1)

print("Server is waiting for connection....")

conn, addr = server_socket.accept()

print("Connection with", addr)

data = conn.recv(1024).decode()

# Check palindrome
if data == data[::-1]:

    num = int(data)

    if num % 2 == 0:
        result = "It is palindrome and even"

    else:
        result = "It is palindrome and odd"

else:
    result = "It is not palindrome"

conn.send(result.encode())

conn.close()

server_socket.close()
```

CLIENT CODE:

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client_socket.connect(("localhost", 18000))

num = input("Enter the number: ")

client_socket.send(num.encode())

result = client_socket.recv(1024).decode()
```

```
print("Server response:", result)
```

```
client_socket.close()
```

3. Fibonacci series-

- **Generate Fibonacci up to N terms**
- **Find sum of Fibonacci series**
- **Display even and odd Fibonacci numbers**

Display the sum of even numbers as evensum and odd number as oddsum

SERVER CODE:

```
import socket
```

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
server_socket.bind(("localhost", 4446))
```

```
server_socket.listen(1)
```

```
print("Server is waiting for connection....")
```

```
conn, addr = server_socket.accept()
```

```
print("Connection with", addr)
```

```
data = conn.recv(1024).decode()
```

```
def fibonacci(n):
```

```
    series = []
```

```
    a, b = 0, 1
```

```
    for i in range(n):
```

```
        series.append(a)
```

```
        a, b = b, a + b
```

```

        return series

n = int(data)

fib_series = fibonacci(n)

# Sum of Fibonacci series
total_sum = sum(fib_series)

# Even and odd Fibonacci numbers
even_nums = []
odd_nums = []

for num in fib_series:

    if num % 2 == 0:

        even_nums.append(num)

    else:

        odd_nums.append(num)

# Sum of even and odd numbers
even_sum = sum(even_nums)
odd_sum = sum(odd_nums)

result = (

    "Fibonacci Series = " + str(fib_series) +

    "\nSum of Series = " + str(total_sum) +

    "\nEven Fibonacci Numbers = " + str(even_nums) +

    "\nOdd Fibonacci Numbers = " + str(odd_nums) +

    "\nEvenSum = " + str(even_sum) +

    "\nOddSum = " + str(odd_sum)

)

conn.send(result.encode())

```

```
conn.close()
```

```
server_socket.close()
```

CLIENT CODE:

```
import socket
```

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client_socket.connect(("localhost", 4446))
```

```
n = input("Enter number of terms: ")
```

```
client_socket.send(n.encode())
```

```
data = client_socket.recv(1024).decode()
```

```
print(data)
```

```
client_socket.close()
```

4. CRC- Data-10011101; Divisor – 1001; Reminder -100

SERVER CODE:

```
import socket
```

```
server_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
```

```
server_socket.bind(("localhost",4446))
```

```
server_socket.listen(1)
```

```
print("server is waiting for connection....")
```

```
conn,addr=server_socket.accept()
```

```
print("connection with",addr)
```

```
data=conn.recv(1024).decode()
```

```
div=conn.recv(1024).decode()
```

```
data=data+"0"*(len(div)-1)
```

```
data=list(data)
```

```
div=list(div)
```

```

for i in range(len(data)-len(div)+1):
    if data[i]=='1':
        for j in range(len(div)):
            if data[i+j]==div[j]:
                data[i+j]='0'
            else:
                data[i+j]='1'
        rem=""
    for i in range(len(div)-1):
        rem=rem+data[-(len(div)-1)+i]
    conn.send(rem.encode())

```

CLIENT CODE:

```

import socket

client_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

client_socket.connect(("localhost",4446))

data=input("enter the data:")

client_socket.send(data.encode())

div=input("enter the divisor")

client_socket.send(div.encode())

rem=client_socket.recv(1024).decode()

print(rem)

```

5. Byte stuffing ↔ original data

- 1) Given the output after byte-stuffing: FLAG A B ESC ESC C ESC ESC ESC FLAG ESC FLAG D FLAG what is the original data? ANS: A B ESC C ESC FLAG FLAG D

A B ESC C ESC FLAG FLAG D-After stuffing, we get A B ESC ESC C ESC ESC ESC FLAG ESC FLAG D

SERVER CODE:

```

import socket

server_socket = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

server_socket.bind(("localhost",17000))

server_socket.listen(1)

print("Server is waiting for connection")

conn,addr = server_socket.accept()

data = conn.recv(1024).decode()

```

```

words = data.split()

# Check whether data is already stuffed
if words[0]=="FLAG" and words[-1]=="FLAG":

    original = []

    i = 1

    while i < len(words)-1:

        if words[i]=="ESC":

            original.append(words[i+1])

            i = i + 2

        else:

            original.append(words[i])

            i = i + 1

    result = "Original Data: " + " ".join(original)
else:

    res = "FLAG "

    for ch in words:

        if ch=="FLAG" or ch=="ESC":

            res = res + "ESC "

        res = res + ch + " "

    res = res + "FLAG"

    result = "Stuffed Data: " + res

conn.send(result.encode())

conn.close()

server_socket.close()

```

CLIENT CODE:

```

import socket

client_socket = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

client_socket.connect(("localhost",17000))

data = input("Enter data: ")

client_socket.send(data.encode())

result = client_socket.recv(1024).decode()

```

```
print(result)

client_socket.close()
```

6. Bit Stuffing ↔ original data

011110111110111110, after bit stuffing output is 011110111110011111010

Server code:

```
import socket

server_socket = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

server_socket.bind(("localhost",18000))

server_socket.listen(1)

print("Server is waiting")

conn,addr = server_socket.accept()

data = conn.recv(1024).decode()

# Destuff

count = 0

original = ""

i = 0

while i < len(data):

    original += data[i]

    if data[i] == '1':

        count += 1

        if count == 5:

            i += 1

            count = 0

        else:

            count = 0

            i += 1

# Restuff to verify

count = 0

check = ""

for bit in original:

    check += bit

    if bit == '1':

        count += 1
```

```

        if count == 5:

            check += '0'

            count = 0

        else:

            count = 0

    if check == data:

        result = "Original Bits: " + original

    else:

        count = 0

        stuffed = ""

        for bit in data:

            stuffed += bit

            if bit == '1':

                count += 1

                if count == 5:

                    stuffed += '0'

                    count = 0

            else:

                count = 0

        result = "Stuffed Bits: " + stuffed

    conn.send(result.encode())

    conn.close()

    server_socket.close()

```

client code:

```

import socket

client_socket = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

client_socket.connect(("localhost",18000))

data = input("Enter bit structure: ")

client_socket.send(data.encode())

print(client_socket.recv(1024).decode())

```

```
client_socket.close()
```

7. FTP (Note: create a new file named send.txt and store some sentence in it, and create another file named recv.txt and store nothing in it-keep it empty)

Server code:

```
import socket

server_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

server_socket.bind(("localhost",18000))

server_socket.listen(1)

print("server is waiting for connection....")

conn,addr=server_socket.accept()

print("connection with",addr)

data=conn.recv(1024)

f=open("recv.txt","wb")

while data:

    f.write(data)

    data=conn.recv(1024)

f.close()

conn.close()

server_socket.close()

print("File received")
```

Client code:

```
import socket

client_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

client_socket.connect(("localhost",18000))

f=open("send.txt","rb")

data=f.read(1024)

while data:

    client_socket.send(data)

    data=f.read(1024)

f.close()

client_socket.close()

print("File sent")
```

8. Display the contents of the requested file; display word count and line count.

Server code:

(NOTE: Create a file sample.txt and save 2-3 sentences, during the output enter the name of the file : sample.txt)

```
import socket

server_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

server_socket.bind(("localhost",18000))

server_socket.listen(1)

print("server is waiting for connection....")

conn,addr=server_socket.accept()

print("connection with",addr)

filename=conn.recv(1024).decode()

f=open(filename,"r")

data=f.read()

words=len(data.split())

lines=len(data.splitlines())

result="File contents:\n"+data+"\n\nWord count:"+str(words)+"\nLine count:"+str(lines)

conn.send(result.encode())

f.close()

conn.close()

server_socket.close()
```

Client code:

```
import socket

client_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

client_socket.connect(("localhost",18000))

filename=input("Enter file name:")

client_socket.send(filename.encode())

result=client_socket.recv(1024).decode()

print(result)

client_socket.close()
```

9. Write a Python socket program to store and fetch student details such as USN, Name, and Marks from a SQLite database.

- Display first three topper details out of 10 entries
- Percentage calculation

-Multiple student entries**Server code:**

(Note: create a new file named test.db and store nothing in it)

```
import socket

import sqlite3

server_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

server_socket.bind(("localhost",5500))

server_socket.listen(1)

print("Server is waiting...")
```

```
conn,addr=server_socket.accept()
```

```
query=conn.recv(1024).decode()
```

```
db=sqlite3.connect("test.db")
```

```
cur=db.cursor()
```

```
cur.execute(query)
```

```
rows=cur.fetchall()
```

```
result=str(rows)
```

```
conn.send(result.encode())
```

```
db.close()
```

```
conn.close()
```

```
server_socket.close()
```

client code:

```
import socket
```

```
client_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
```

```
client_socket.connect(("localhost",5500))
```

```
query=input("Enter query:")
```

```
client_socket.send(query.encode())
```

```
data=client_socket.recv(1024).decode()
```

```
print("Result:",data)
```

```
client_socket.close()
```

db code:

```
import sqlite3
```

```
conn=sqlite3.connect("test.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""
```

```
create table if not exists students(
```

```
USN text,
```

```
Name text,
```

```
Marks int
```

```
)
```

```
""")
```

```
cur.execute("insert into students values('4SU22IS001','Asha',95)")
```

```
cur.execute("insert into students values('4SU22IS002','Ravi',88)")
```

```
cur.execute("insert into students values('4SU22IS003','Priya',91)")
```

```
cur.execute("insert into students values('4SU22IS004','Neha',85)")
```

```
cur.execute("insert into students values('4SU22IS005','Kiran',98)")
```

```
cur.execute("insert into students values('4SU22IS006','Anu',90)")
```

```
cur.execute("insert into students values('4SU22IS007','Rahul',87)")
```

```
cur.execute("insert into students values('4SU22IS008','Megha',93)")
```

```
cur.execute("insert into students values('4SU22IS009','Arjun',80)")
```

```
cur.execute("insert into students values('4SU22IS010','Sneha',89)")
```

```
conn.commit()
```

```
conn.close()
```

queries:

Display all students

```
select * from students;
```

First 3 toppers

```
select * from students order by Marks desc limit 3;
```

Percentage calculation

(assuming marks out of 100)

```
select USN,Name,Marks,(Marks*100)/100 as Percentage from students;
```

Multiple student entries

Already inserted 10 entries in db.py

10. Develop a client-server application using Python sockets to store and retrieve employee details such as Employee ID, Name, Department, and Salary (Basic+DA+HRA+CCA) from a database.

- **Department-wise employee display**
- **Search highest salary employee**

Salary increment calculation(3% of the basic)

NOTE: Server and client code remains same as 9th question. (Note: create a new file named emp.db and store nothing in it)

DB code:

```
import sqlite3
```

```
conn=sqlite3.connect("emp.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""
```

```
create table if not exists employee(
```

```
EmplID int,
```

```
Name text,
```

```
Department text,
```

```
Basic int,
```

```
DA int,
```

```
HRA int,
```

```
CCA int
```

```
)
```

```
""")
```

```
cur.execute("insert into employee values(1,'Asha','HR',30000,5000,4000,1000)")
```

```
cur.execute("insert into employee values(2,'Ravi','IT',40000,6000,5000,1500)")
```

```
cur.execute("insert into employee values(3,'Neha','HR',35000,5500,4500,1200)")
cur.execute("insert into employee values(4,'Kiran','Sales',45000,7000,6000,2000)")
cur.execute("insert into employee values(5,'Anu','IT',38000,5000,4000,1000)")

conn.commit()

conn.close()
```

queries:

Display all employees

```
select * from employee;
```

Department-wise employee display

Example for IT:

```
select * from employee where Department='IT';
```

Example for HR:

```
select * from employee where Department='HR';
```

Highest salary employee

```
select EmpID,Name,Department,
(Basic+DA+HRA+CCA) as Salary
from employee
order by Salary desc
limit 1;
```

Salary increment calculation (3% of Basic)

```
select EmpID,Name,Basic,
(Basic*3)/100 as Increment
from employee;
```

11.Implement a socket-based Python program to store and fetch book details such as Book ID, Book Name, Author, and Price from a database.

- **Search by author**

-Sort books by price

NOTE: Server and client code remains same as 9th question. (Note: create a new file named book.db and store nothing in it)

DB CODE:

```
import sqlite3
```

```
conn=sqlite3.connect("book.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""  
create table if not exists books(  
BookID int,  
BookName text,  
Author text,  
Price int  
)  
""")
```

```
cur.execute("insert into books values(1,'Python Basics','Ravi',450)")
```

```
cur.execute("insert into books values(2,'DBMS','Anu',600)")
```

```
cur.execute("insert into books values(3,'Networking','Ravi',550)")
```

```
cur.execute("insert into books values(4,'AI Concepts','Neha',700)")
```

```
cur.execute("insert into books values(5,'Web Dev','Anu',500)")
```

```
conn.commit()
```

```
conn.close()
```

Queries:

Display all books

```
select * from books;
```

Search by author

```
select * from books where Author='Ravi';
```

or

```
select * from books where Author='Anu';
```

Sort books by price (ascending)

```
select * from books order by Price;
```

Sort books by price (descending)

```
select * from books order by Price desc;
```

12. Create a Python TCP socket application to store and retrieve customer details such as Customer ID, Name, Phone Number, and Address from a database.

- **Display Customers from Particular City**
- **Count Total Customers**
- **Sort Customers Alphabetically**
- **Search Customers Starting with Letter**

Find Customers with Same Address

NOTE: Server and client code remains same as 9th question. (Note: create a new file named customer.db and store nothing in it)

DB CODE:

```
import sqlite3

conn=sqlite3.connect("customer.db")

cur=conn.cursor()

cur.execute("""
create table if not exists customer(
CustomerID int,
Name text,
PhoneNo text,
Address text
)
""")

cur.execute("insert into customer values(1,'Asha','9876543210','Mysore')")
cur.execute("insert into customer values(2,'Ravi','9876543211','Hubli')")
cur.execute("insert into customer values(3,'Neha','9876543212','Mysore')")
cur.execute("insert into customer values(4,'Arjun','9876543213','Dharwad')")
cur.execute("insert into customer values(5,'Anu','9876543214','Hubli')")
```

```
conn.commit()
```

```
conn.close()
```

queries:

Display all customers

```
select * from customer;
```

Customers from particular city

```
select * from customer where Address='Mysore';
```

```
select * from customer where Address='Hubli';
```

Count total customers

```
select count(*) from customer;
```

Sort customers alphabetically

```
select * from customer order by Name;
```

Search customers starting with letter

```
select * from customer where Name like 'A%';
```

```
select * from customer where Name like 'R%';
```

Customers with same address

```
select Address,count(*)  
from customer  
group by Address  
having count(*)>1;
```

13. Implement a Python socket application to store and fetch railway reservation details such as Passenger Name, Train Number, Seat Number, and Journey Date from a database.

- **Search Passenger by Name**
- **Update Seat Number**
- **Cancel Reservation**
- **Display Reservations Train-Wise**
- **Display Confirmed Tickets**

Display Waiting List Passengers

NOTE: Server and client code remains same as 9th question. (Note: create a new file named railway.db and store nothing in it)

DB CODE:

```
import sqlite3

conn=sqlite3.connect("railway.db")

cur=conn.cursor()

cur.execute("""
create table if not exists reservation(
PassengerName text,
TrainNo int,
SeatNo text,
JourneyDate text,
Status text
)
""")

cur.execute("insert into reservation values('Asha',101,'S1-12','2026-05-28','Confirmed')")
cur.execute("insert into reservation values('Ravi',101,'S1-13','2026-05-28','Confirmed')")
cur.execute("insert into reservation values('Neha',102,'WL-1','2026-05-29','Waiting')")
cur.execute("insert into reservation values('Arjun',101,'WL-2','2026-05-28','Waiting')")
cur.execute("insert into reservation values('Anu',103,'S2-10','2026-05-30','Confirmed')")

conn.commit()

conn.close()
```

Queries:

Display all reservations

```
select * from reservation;
```

Search passenger by name

```
select * from reservation where PassengerName='Asha';
```

Update seat number

```
update reservation
set SeatNo='S1-20'
where PassengerName='Ravi';
```

Cancel reservation

```
delete from reservation
where PassengerName='Arjun';
```

Display reservations train-wise

Example train 101:

```
select * from reservation
where TrainNo=101;
```

Display confirmed tickets

```
select * from reservation
where Status='Confirmed';
```

Display waiting list passengers

```
select * from reservation
where Status='Waiting';
```

14. Write a Python socket program to store and retrieve bank account details such as Account Number, Customer Name, Account Type, and Balance from a database

- **Search Account by Account Number**
- **Deposit Amount**
- **Withdraw Amount**
- **Minimum Balance Check**
- **Find Highest and Lowest Balance Account**

Sort Customers by Balance

NOTE: Server and client code remains same as 9th question. (Note: create a new file named bank.db and store nothing in it)

DB CODE:

```
import sqlite3

conn=sqlite3.connect("bank.db")

cur=conn.cursor()

cur.execute("""

create table if not exists bank(

AccountNo int,

CustomerName text,
```

```
AccountType text,  
Balance int  
)  
""")
```

```
cur.execute("insert into bank values(1001,'Asha','Savings',50000)")  
cur.execute("insert into bank values(1002,'Ravi','Current',25000)")  
cur.execute("insert into bank values(1003,'Neha','Savings',80000)")  
cur.execute("insert into bank values(1004,'Arjun','Current',15000)")  
cur.execute("insert into bank values(1005,'Anu','Savings',95000)")
```

```
conn.commit()
```

```
conn.close()
```

Queries:

Display all accounts

```
select * from bank;
```

Search account by account number

```
select * from bank  
where AccountNo=1001;
```

Deposit amount

```
update bank  
set Balance=Balance+5000  
where AccountNo=1001;
```

Withdraw amount

```
update bank  
set Balance=Balance-2000  
where AccountNo=1001;
```

Minimum balance check

(Example minimum balance = 10000)

```
select * from bank  
where Balance<10000;
```

Highest balance account

```
select * from bank  
order by Balance desc  
limit 1;
```

Lowest balance account

```
select * from bank  
order by Balance  
limit 1;
```

Sort customers by balance

Ascending:

```
select * from bank  
order by Balance;
```

Descending:

```
select * from bank  
order by Balance desc;
```

15.Implement a socket-based examination result system using Python to store and retrieve student result details such as Student Name, Subject, Marks, and Grade from a database.

- **Update Student Marks**
- **Calculate Total Marks**
- **Find Highest and lowest Marks**

Find Subject Topper

NOTE: Server and client code remains same as 9th question. (Note: create a new file named results.db and store nothing in it)

DB CODE:

```
import sqlite3  
  
conn=sqlite3.connect("result.db")  
  
cur=conn.cursor()  
  
cur.execute("""  
create table if not exists result(  
StudentName text,  
Subject text,  
Marks int,  
Grade text  
)  
""")
```

```
cur.execute("insert into result values('Asha','Maths',95,'A')")
cur.execute("insert into result values('Ravi','Maths',88,'B')")
cur.execute("insert into result values('Neha','Science',92,'A')")
cur.execute("insert into result values('Arjun','Science',75,'C')")
cur.execute("insert into result values('Anu','Maths',98,'A')")
```

```
conn.commit()
```

```
conn.close()
```

Queries:

Display all results

```
select * from result;
```

Update student marks

Example:

```
update result
set Marks=90
where StudentName='Ravi';
```

Calculate total marks

```
select sum(Marks)
from result;
```

Highest marks

```
select *
from result
order by Marks desc
limit 1;
```

Lowest marks

```
select *
from result
order by Marks
limit 1;
```

Subject topper

Example for Maths:

```
select *
from result
where Subject='Maths'
order by Marks desc
limit 1;
```

Example for Science:

```
select *
from result
where Subject='Science'
```

order by Marks desc
limit 1;